

Recruiter Backend API and Flow Contract

Single backend handoff for recruiter signup, email verification, organization review, login, recruiter workspace APIs, candidate access, credit purchases, invoices, and hospital profile management.

Proposed contract — backend team to confirm

Purpose: Use this document to agree on the complete recruiter journey and the API contract before backend implementation and frontend end-to-end testing. Endpoint paths and response formats below are proposed and must be confirmed against the backend.

Project: MedHire

Audience: Backend engineering and frontend integration

Prepared: September 24, 2026

Related frontend: `src/lib/authApi.js`, `src/lib/recruiterAuthApi.js`, `src/lib/recruiterApi.js`

1. Scope and Flow Overview

1. Register Create recruiter and organization; email verification is required.	2. Verify email Consume registration code; keep organization review separate.	3. Sign in Return authenticated recruiter session and organization status.
4. Organization review Pending/rejected accounts cannot read recruiter resources.	5. Approved workspace Check existing job posts; post openings; search candidates, shortlist, unlock, manage credits/profile.	6. Payments Hosted checkout; verified provider webhook records payment and credits.

This contract includes recruiter authentication and recruiter workspace data. Shared candidate authentication is outside scope. Frontend route guards are for navigation only; the backend is the authorization boundary.

2. Common API Rules

Item	Requirement
Base URL	Frontend reads <code>VITE_API_BASE_URL</code> ; it defaults to <code>/api</code> . Paths here are relative to that base.
Encoding	JSON request/response bodies use <code>application/json</code> . List endpoints return a consistent envelope.
Role	Use canonical role <code>recruiter</code> . Reject candidate or unknown roles from recruiter-only requests.
Identity and scope	Resolve recruiter and organization from the authenticated session/token; never trust a client-supplied organization ID for authorization.
Time and money	Use ISO-8601 UTC timestamps. Represent money as decimal string plus ISO-4217 currency. Do not calculate money in binary floating point.
Errors	Use the shared error format in section 12. Provide stable codes and field-level errors where applicable.
Security	Hash passwords, rate-limit auth/OTP, redact secrets, scope candidate/payment/profile data to organization, and enforce approval on every recruiter resource route.
Pagination	List defaults: <code>page=1</code> , <code>pageSize=20</code> ; clamp <code>pageSize</code> to 100.

```
{
  "items": [],
  "pagination": { "page": 1, "pageSize": 20, "total": 0, "totalPages": 0 }
}
```

3. Recruiter Registration

POST /auth/register

```
{
  "role": "recruiter",
  "fullName": "Rahul Menon",
  "email": "rahul@hospital.com",
  "phone": "+919876543210",
  "designation": "HR Manager",
  "organizationName": "MedCare Hospital",
  "organizationWebsite": "https://hospital.com",
  "city": "Chennai",
  "state": "Tamil Nadu",
  "password": "ExamplePassword@123",
  "termsAccepted": true
}
```

The current UI collects a ten-digit phone number and prepends +91. Confirm whether country code must be selectable. Password confirmation is checked in browser and is not sent.

Backend behavior: Validate all fields; normalize email; create recruiter and organization with email unverified and verification status pending; send registration OTP; return **201 Created**. Do not grant recruiter workspace access at registration.

```
{
  "message": "Verification code sent",
  "userId": "user_123",
  "email": "rahul@hospital.com",
  "role": "recruiter",
  "emailVerified": false,
  "organizationVerificationStatus": "pending"
}
```

4. Email Verification and Resend

POST /auth/verify-code

```
{ "email": "rahul@hospital.com", "code": "482916", "purpose": "registration" }
```

On success, consume the one-time code and mark the email verified. This must not approve the organization.

```
{
  "verified": true,
  "purpose": "registration",
  "user": {
    "id": "user_123",
    "role": "recruiter",
    "email": "rahul@hospital.com",
    "emailVerified": true,
    "organizationVerificationStatus": "pending"
  }
}
```

POST /auth/resend-code

```
{ "email": "rahul@hospital.com", "purpose": "registration" }
```

OTP is six numeric digits, expires in about 10 minutes, single-use, and invalidated after success. Rate-limit by account and IP (suggested resend cooldown 60 seconds), and limit verification attempts.

5. Recruiter Login and Access State

POST /auth/login

```
{ "email": "rahul@hospital.com", "password": "ExamplePassword@123", "rememberMe": true }
```

rememberMe is a session-duration preference, not a user profile field. Successful login returns session information and the current authoritative organization status.

```
{
  "accessToken": "eyJ...",
  "refreshToken": "refresh-token-value",
  "expiresIn": 3600,
  "user": {
    "id": "user_123",
    "role": "recruiter",
    "fullName": "Rahul Menon",
    "email": "rahul@hospital.com",
    "emailVerified": true,
    "organizationId": "org_456",
    "organizationName": "MedCare Hospital",
    "organizationVerificationStatus": "pending"
  }
}
```

State	Frontend route behavior	Backend behavior
Email unverified	On the <code>EMAIL_NOT_VERIFIED</code> error, route to registration email verification; user enters the existing code or requests a resend. Do not store a session.	Return <code>403 Forbidden</code> with code <code>EMAIL_NOT_VERIFIED</code> ; do not issue access or refresh tokens.
Organization pending	Show verification pending page; block recruiter workspace.	Return <code>organizationVerificationStatus: "pending"</code> ; resource APIs return 403.
Organization approved	Open recruiter dashboard and workspace.	Return <code>"approved"</code> ; authorize only this organization's data.
Organization rejected	Show rejection/support/resubmission UX (currently needs frontend decision).	Return <code>"rejected"</code> and optionally a safe, user-facing reason.
Wrong role / invalid credentials	Show form error; do not store a mismatched session.	Use generic credential errors where needed to avoid account enumeration.

When organization approval changes, the next login/session refresh must return the latest state. Protect every resource with current backend status even when browser session data is stale.

6. Password Recovery

Use shared auth endpoints while preserving recruiter context in the UI. Do not reveal whether the email exists.

POST /auth/forgot-password

```
{ "email": "rahul@hospital.com" }
```

Return generic [202 Accepted](#) for both known and unknown emails.

POST /auth/verify-code

```
{ "email": "rahul@hospital.com", "code": "482916", "purpose": "password_reset" }
```

On success return a short-lived, single-use reset token. Keep this purpose distinct from registration verification.

POST /auth/reset-password

```
{ "email": "rahul@hospital.com", "resetToken": "reset-token-value", "newPassword": "NewExamplePassword@123" }
```

Invalidate reset token and revoke sessions according to security policy. Confirmation password is frontend-only.

7. Recruiter Workspace API

Every route in this section requires an approved recruiter access token:

```
Authorization: Bearer <accessToken>
Accept: application/json
Content-Type: application/json
```

Return **403 ORGANIZATION_NOT_APPROVED** for pending/rejected organizations. Derive the organization from token claims or server-side session.

GET /recruiter/dashboard

```
{
  "metrics": { "candidateSearches": 284, "profilesUnlocked": 47,
    "shortlistedCandidates": 23, "availableCredits": 156 },
  "organization": { "name": "MedCare Hospital", "location": "Chennai, IN",
    "verificationStatus": "approved" },
  "recentActivity": [
    { "id": "act_01", "description": "Candidate profile unlocked",
      "occurredAt": "2026-09-24T09:00:00Z" }
  ],
  "recommendedCandidates": []
}
```

Recommended profiles must follow the masking rules below.

GET /recruiter/jobs

List jobs posted by the authenticated recruiter's organization. Query parameters: `page`, `pageSize`, and optionally `status`. Return the common list envelope, including an empty `items` array and zero totals when the account has no postings. Each item includes `id`, `title`, `specialty`, `location`, `employmentType`, optional `salaryRange`, `description`, `requirements` (string array), `status`, `postedAt`, and optional `updatedAt`.

After approved recruiter login, the frontend calls this endpoint: a non-empty result routes to `/recruiter/dashboard`; no jobs routes to `/recruiter/jobs` to create the first opening. If this check fails, login continues to the dashboard and the Job Postings page displays its own error. Scope results to the authenticated organization; do not use a client-supplied organization ID.

POST /recruiter/jobs

```
{
  "title": "Registered Nurse - Critical Care",
  "specialty": "Critical Care",
  "location": "Chennai, Tamil Nadu",
  "employmentType": "Full-time",
  "salaryRange": "₹8,00,000-₹12,00,000 / year",
  "description": "Provide direct care to patients in the critical care unit.",
  "requirements": ["Active state nursing license", "Two years of ICU experience"]
}
```

`salaryRange` is optional; current form requires the other fields. Validate and trim values, enforce length limits, validate employment type, and sanitize rendered text. Enforce the organization's job-post plan/entitlement on the server. Create the posting under the authenticated organization, set `postedAt`, initialize `status` to `published` (or return the moderation state), and return **201 Created** with the created job object.

Return **422 VALIDATION_ERROR** with field errors for invalid values and **409 JOB_POST_LIMIT_REACHED** when the account has reached its entitlement. Do not create a record on either failure. This initial frontend supports listing and creating; update/close/delete actions are not exposed.

```
{
  "id": "job_123",
  "title": "Registered Nurse - Critical Care",
  "specialty": "Critical Care",
  "location": "Chennai, Tamil Nadu",
  "employmentType": "Full-time",
  "salaryRange": "₹8,00,000-₹12,00,000 / year",
  "description": "Provide direct care to patients in the critical care unit.",
  "requirements": ["Active state nursing license", "Two years of ICU experience"],
  "status": "published",
  "postedAt": "2026-10-06T08:00:00Z"
}
```

GET /recruiter/candidates

Query: `query`, `specialty`, `location`, `minExperience`, `maxExperience`, `licenseState`, `employment`, `availability`, `page`, `pageSize`.

Return the common list envelope. Search item:

```
{ "id":"MH-10482", "initials":"MH", "title":"Registered Nurse",
  "specialty":"Critical Care", "experience":"5 Years Experience",
  "experienceYears":5, "location":"Chicago, IL", "education":"BSN",
  "license":"Active License - IL", "availability":"Immediate",
  "employment":"Full-time", "tags":["ICU","BLS","ACLS"], "unlocked":false }
```

Search results must not return full name, personal email, phone, address, resume URL, or other private profile details before access is unlocked.

GET /recruiter/candidates/{candidateId}

Return candidate summary, profile sections, access state, and credit balance. Contact fields remain null/masked until the organization has unlocked that candidate.

```
{ "candidate": {}, "profile": { "summary":"...", "workExperience":[],
  "skills":[], "career":[], "contact": { "fullName":null, "email":null,
    "phone":null, "address":null } },
  "access": { "unlocked":false, "unlockCost":5 },
  "creditBalance": { "available":156 } }
```

POST /recruiter/candidates/{candidateId}/unlock

```
Request: {}
Response: { "candidateId": "MH-10482", "unlocked": true, "creditsUsed": 5,
  "creditBalance": { "available": 151 }, "profile": {} }
```

Atomically verify current organization approval and balance, debit once, record purchase, and grant access. Support an **Idempotency-Key** header so safe retries cannot double-charge. Return **409 INSUFFICIENT_CREDITS** with no debit when balance is low.

GET /recruiter/purchased-candidates

Query: **query**, **specialty**, **location**, **status**, **page**, **pageSize**. Return common list envelope. Each result belongs to this organization and includes **id**, **initials**, **name**, **title**, **specialty**, **experience**, **location**, **unlockedDate**, **status** (Active/Contacted). Only expose resume/contact access after purchase; deliver downloads with an authorized short-lived URL.

GET /recruiter/shortlisted-candidates

Query: **query**, **specialty**, **unlockStatus**, **location**, **page**, **pageSize**. Return common list envelope. Candidate name stays masked while locked.

POST /recruiter/shortlisted-candidates

```
{ "candidateId": "MH-10482" }
```

Return **201** (or **200** if already saved): { "candidateId": "MH-10482", "shortlisted": true }.

DELETE /recruiter/shortlisted-candidates/{candidateId}

Return **204 No Content**. Remove only the shortlist association; never delete the candidate or unlock purchase.

GET /recruiter/plans

Return { "plans": [], "currentPlan": {} }. Plan fields: **id**, **name**, **price**, **currency**, **billingInterval**, **credits**, **jobPosts**, **recruiterSeats**, **validityDays**, **features**. Server is authoritative for price and entitlements.

GET /recruiter/credits

```
{ "available": 156, "used": 94, "currency": "USD" }
```

8. Checkout, Payment, and Invoice

Payment security: Do not send raw card number, expiry, or CVV to MedHire API. Use provider-hosted checkout or provider tokenization. A browser redirect must never mark a payment successful or add credits.

POST /recruiter/checkout

```
{ "planId":"professional",
  "successUrl":"https://app.example.com/recruiter/payment-success",
  "cancelUrl":"https://app.example.com/recruiter/payment-failed" }
```

Return **201 Created** with provider session and redirect URL:

```
{ "checkoutSessionId":"cs_123",
  "checkoutUrl":"https://payment-provider.example/session/cs_123" }
```

Validate plan and price server-side. A verified provider webhook is the source of truth for transaction status and credit grant. Use idempotent webhook handling; failed/cancelled checkout leaves balance unchanged.

GET /recruiter/checkout/{checkoutSessionId}

Return the authenticated organization's server-verified checkout status: **pending**, **successful**, **failed**, or **cancelled**. Return transaction ID and safe user-facing message when available. Scope the session to the owning organization. Update success only from a verified payment provider webhook, never from the browser redirect.

GET /recruiter/payment-history

Query: **status** (Successful/Failed/Pending), **startDate**, **endDate**, **page**, **pageSize**. Return common list envelope.

```
{ "id":"TXN-2026-09187", "date":"2026-09-18T00:00:00Z",
  "plan":"Professional", "credits":50, "amount":"268.92", "currency":"USD",
  "method":"Visa ending in 4242", "status":"Successful",
  "invoiceAvailable":true }
```

GET /recruiter/payment-history/{transactionId}/invoice

Authorize transaction ownership and invoice eligibility. Return invoice details or a short-lived authorized PDF URL/stream. Never expose public permanent invoice URLs.

```
{ "invoice": { "invoiceNumber":"INV-2026-1001", "date":"2026-09-18",
  "billedTo":{}, "billedFrom":{}, "lineItems":[], "subtotal":"250.00",
  "tax":"18.92", "total":"268.92", "currency":"USD",
  "transactionId":"TXN-2026-09187", "paymentMethod":"Visa ending in 4242",
  "status":"Successful" } }
```

9. Hospital Profile

GET /recruiter/hospital-profile

Return the authenticated organization's editable profile, read-only review status, and hiring preferences:

```
{ "profile": { "hospitalName":"MedCare Hospital", "organizationType":"Hospital",
  "registrationNumber":"REG-123", "website":"https://hospital.com",
  "contactEmail":"hr@hospital.com", "hospitalPhone":"+919876543210",
  "address":"...", "city":"Chennai", "state":"Tamil Nadu", "postal":"600001",
  "country":"India", "recruiterName":"Rahul Menon", "designation":"HR Manager",
  "businessEmail":"rahul@hospital.com", "recruiterPhone":"+919876543210" },
  "verification": { "status":"approved", "verifiedAt":"2026-09-20T12:00:00Z" },
  "hiringPreferences": { "specialties":[], "locations":[], "employmentTypes":[] } }
```

PATCH /recruiter/hospital-profile

Accept { "profile": { ...editableFields }, "hiringPreferences": { "specialties": [], "locations": [], "employmentTypes": [] } }. Do not accept changes to verification status, organization ownership, or approval. Return the updated envelope. Return 422 `VALIDATION_ERROR` with field errors for invalid values. Define whether changes to legal name, registration, or website trigger re-review.

10. Validation Rules

Field/group	Current frontend rule	Backend must enforce
Recruiter name, designation, organization name, city, state	Required, non-empty after trim.	Required, length bounds, normalize whitespace, reject invalid characters as appropriate.
Email	Basic valid email format; normalized lowercase on submit.	Validate, normalize, enforce uniqueness and verification policy.
Phone	Ten numeric digits; UI currently adds +91.	Validate E.164/country policy; confirm whether international numbers are supported.
Website	Optional; if provided, valid http/https URL.	Validate URL and safe scheme. Do not fetch arbitrary URLs without SSRF controls.
Password	8–72 chars, uppercase, number, symbol; confirmation must match.	Enforce password policy and secure hashing; confirmation stays client-side.
Terms	Must be accepted; sent as <code>termsAccepted: true</code> .	Record consent and applicable terms version/time as required.
OTP	Exactly six numeric digits.	Expiry, single use, attempt cap, resend cooldown, anti-enumeration.
Profile updates	Hospital profile UI currently has no field validation; add server validation.	Validate contact values and editable fields; lock legal/verification-owned fields.
Search and filters	Search and filter field choices are UI-controlled.	Validate supported filters, clamp page values, scope query to approved organization permissions.

11. Error Format and Status Codes

```
{ "error": { "code": "VALIDATION_ERROR",  
  "message": "Please correct the highlighted fields.",  
  "fields": { "email": "Enter a valid email address." } } }
```

Status	Meaning / examples
400	Malformed JSON or invalid request shape.
401	Missing/expired token; invalid login/reset credentials.
403	Wrong role, unverified email, pending/rejected organization, or unauthorized action.
404	Resource unavailable or not visible to this organization.
409	Email conflict, insufficient credits, or conflicting state.
422	Field validation/business rule failure.
429	Login, OTP, resend, or payment rate limit.
500	Unexpected server error; do not return stack traces or secrets.

12. Data Protection and Authorization Requirements

- Candidate records, purchases, shortlists, payments, invoices, and hospital profile reads/writes must be organization-scoped server-side.
- Keep personal candidate details, resume links, and contact information masked until an authorized unlock purchase is confirmed.
- Verify organization approval at action time, especially candidate unlock and checkout.
- Never accept a client-submitted credit balance, plan price, payment success flag, organization ID for access scope, or verification status as authoritative.
- Make credit debits and payment webhook processing transactional and idempotent.
- Use secure transport, password hashing, token expiry/revocation, rate limiting, audit events for sensitive actions, and redacted logs.

13. Frontend Integration Status and Known Gaps

Authentication: Current frontend register/login/OTP/recovery screens call shared auth API methods through recruiter-specific auth mapping. The pending/approved route decision depends on the backend returning the role, token, email verification, and organization verification fields defined here.

Workspace integration: Recruiter workspace screens now request data and actions through `src/lib/recruiterApi.js`, including `GET` and `POST /recruiter/jobs` for the Job Postings page. Recruiter sample records were removed; pages show loading, empty, or API error states instead of inventing records when an endpoint is unavailable. The proposed backend routes must be implemented and confirmed for the complete flow to run.

Checkout now redirects to a hosted payment provider and checks the backend session status on return. Payment success and credit changes must come from verified provider webhooks. Payment history and invoices display only backend records.

14. Backend Delivery and Acceptance Checklist

- Confirm API base URL, route names, request/response schemas, role names, and error codes.
- Implement recruiter registration, OTP email delivery/verification/resend, login, and password recovery.
- Define organization review lifecycle and rejected-account resubmission/support behavior.
- Return `organizationVerificationStatus` and identity fields consistently from login and any session refresh.
- Enforce approved organization status server-side for every recruiter resource/action.
- Implement recruiter job listing and creation; scope postings to the authenticated organization and enforce job-post plan limits.
- Implement dashboard, candidate search/detail/unlock, purchased candidates, shortlist CRUD, plans, credits, hospital profile GET/PATCH.
- Choose payment provider and implement hosted checkout, verified webhooks, idempotency, history, and secure invoice delivery.
- Provide staging URL, CORS configuration, test accounts for pending/approved/rejected states, and representative data.
- Test OTP expiry/replay/rate limits; wrong role; invalid credentials; pending/rejected access; insufficient credits; duplicate unlock retries; cross-organization access; failed payment; duplicate webhook; invoice ownership; profile validation.
- Frontend team verifies every recruiter screen against the confirmed API schemas and acceptance cases before release.

15. Frontend Route and File Reference

Flow	Routes / implementation
Registration	/register/recruiter — src/pages/RegisterRecruiter.jsx
Login and role dispatch	/login?role=recruiter — src/pages/Login.jsx
Email OTP	/verify-email?role=recruiter — src/pages/EmailVerification.jsx
Forgot/reset password	/forgot-password?role=recruiter, /reset-password
Pending review and access guard	/recruiter/verification-pending, src/components/recruiter/RequireRecruiterAccess.jsx
Recruiter data API client	src/lib/recruiterApi.js
Job postings	/recruiter/jobs — src/pages/recruiter/RecruiterJobs.jsx (sidebar: Job Postings)
Related documents	Recruiter API contract source · Shared authentication contract · Recruiter login flow detail

Prepared for the MedHire backend team. Treat endpoint names and schemas as proposed until confirmed. This document describes the required contract; it does not assert that backend endpoints are deployed or that end-to-end flows have passed.